

Tutorial 9: Dummy Variables and Robust Standard Errors

Wenhui Xu

ECO375

Contents

Overview	1
1. Dummy variables and the dummy variable trap	2
1.1 What the problem is asking (plain English)	2
1.2 Step-by-step plan (how to solve it)	2
1.3 Common pitfalls	2
1.4 Data load and quick checks	2
1.5 Model 1: intercept + one dummy	3
1.6 Model 2: no intercept + both dummies (equivalent parameterization)	4
Create the complementary dummy	4
1.7 Model 3: intercept + both dummies (dummy variable trap)	5
2. Robust standard errors (White / HC)	5
2.1 What the problem is asking (plain English)	5
2.2 Step-by-step plan (how to solve it)	5
2.3 Common pitfalls	6
2.4 Module: What are <code>vcovHC()</code> and <code>coeftest()</code> ? (first-time tools)	6
<code>sandwich::vcovHC()</code>	6
<code>lmtest::coeftest()</code>	6
2.5 Robust SE using <code>vcovHC()</code> + <code>coeftest()</code>	6
2.6 HW (complete): compute White (HC0) robust SE “by hand”	7
Wrap-up	7

Overview

This tutorial corresponds to the attached `tutorial_9.R`. It is concept-focused and has two goals:

1. **Dummy variables and the dummy variable trap** (perfect multicollinearity when you include all category dummies plus an intercept).
2. **Robust standard errors (White/HC)** using `sandwich::vcovHC()` and `lmtest::coeftest()`.

We use the `cps91` dataset from the `wooldridge` package.

```
if (!requireNamespace("wooldridge", quietly = TRUE)) install.packages("wooldridge")
if (!requireNamespace("dplyr", quietly = TRUE)) install.packages("dplyr")
if (!requireNamespace("sandwich", quietly = TRUE)) install.packages("sandwich")
if (!requireNamespace("lmtest", quietly = TRUE)) install.packages("lmtest")

library(wooldridge)
library(dplyr)
library(sandwich)
library(lmtest)
```

1. Dummy variables and the dummy variable trap

1.1 What the problem is asking (plain English)

We want to estimate and interpret a regression of husband's earnings (`husearns`) on union status (`husunion`, a binary variable).

Then we want to understand what happens if we include *both* dummies (`husunion` and `nunion`) and whether we include an intercept.

Key idea: if a variable is binary, the regression can be written in different (equivalent) ways—but some specifications create perfect collinearity.

1.2 Step-by-step plan (how to solve it)

- 1) Load `cps91` and inspect the binary regressor.
- 2) Estimate a model with an intercept and one dummy: `husearns ~ husunion`.
- 3) Create the complementary dummy `nunion = 1 - husunion`.
- 4) Estimate a model **without an intercept** using both dummies: `husearns ~ 0 + nunion + husunion`.
- 5) Try estimating a model **with an intercept and both dummies** and see what happens (dummy variable trap).
- 6) Interpret coefficients in each specification.

1.3 Common pitfalls

- **Including all category dummies plus an intercept:** this creates perfect collinearity.
- **Forgetting what the intercept represents:** with a single dummy, the intercept is the mean of the base group.
- **Misreading the “no intercept” formula:** in R, `0 +` removes the intercept.

1.4 Data load and quick checks

```
data("cps91")
cps91 <- as_tibble(cps91)

cps91 %>%
```

```
summarise(
  n = n(),
  pct_union = mean(husunion == 1, na.rm = TRUE),
  mean_husearns = mean(husearns, na.rm = TRUE)
)
```

```
## # A tibble: 1 x 3
##       n pct_union mean_husearns
##   <int>    <dbl>        <dbl>
## 1  5634      0.232          454.
```

1.5 Model 1: intercept + one dummy

Model:

$$husearns = \beta_0 + \beta_1 husunion + u$$

Interpretation: - β_0 : average earnings for husbands **not** in a union - β_1 : difference in average earnings (union – nonunion)

```
lm1 <- lm(husearns ~ husunion, data = cps91)
summary(lm1)
```

```
##
## Call:
## lm(formula = husearns ~ husunion, data = cps91)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -595.92 -240.92  -68.92  149.08 1307.08
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  615.9243     6.2634  98.337  <2e-16 ***
## husunion      0.4087     12.9924   0.031   0.975
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 353.4 on 4146 degrees of freedom
## (1486 observations deleted due to missingness)
## Multiple R-squared:  2.386e-07, Adjusted R-squared:  -0.000241
## F-statistic: 0.0009894 on 1 and 4146 DF, p-value: 0.9749
```

```
coef(lm1)
```

```
## (Intercept)    husunion
## 615.9243090    0.4086785
```

A quick way to verify the interpretation is to compute group means directly:

```
cps91 %>%
  group_by(husunion) %>%
  summarise(
    mean_husearns = mean(husearns, na.rm = TRUE),
    n = n()
  )
```

```
## # A tibble: 3 x 3
##   husunion mean_husearns      n
##   <int>         <dbl> <int>
## 1      0           616.  3184
## 2      1           616.   964
## 3     NA            0  1486
```

1.6 Model 2: no intercept + both dummies (equivalent parameterization)

Create the complementary dummy

```
cps91 <- cps91 %>% mutate(nounion = 1 - husunion)

# sanity check: nounion + husunion should be 1 for all observations
cps91 %>% summarise(all_one = all(nounion + husunion == 1))
```

```
## # A tibble: 1 x 1
##   all_one
##   <lgl>
## 1 NA
```

Model:

$$husearns = \gamma_0 nounion + \gamma_1 husunion + u$$

(with **no** intercept)

Interpretation: - γ_0 : mean earnings for nonunion husbands - γ_1 : mean earnings for union husbands

```
lm2 <- lm(husearns ~ 0 + nounion + husunion, data = cps91)
summary(lm2)
```

```
##
## Call:
## lm(formula = husearns ~ 0 + nounion + husunion, data = cps91)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -595.92 -240.92  -68.92  149.08 1307.08
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## nounion      615.924      6.263   98.34  <2e-16 ***
## husunion     616.333     11.383   54.15  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 353.4 on 4146 degrees of freedom
## (1486 observations deleted due to missingness)
## Multiple R-squared:  0.7524, Adjusted R-squared:  0.7523
## F-statistic: 6301 on 2 and 4146 DF, p-value: < 2.2e-16
```

```
coef(lm2)
```

```
##   nounion husunion
## 615.9243 616.3330
```

1.7 Model 3: intercept + both dummies (dummy variable trap)

If you include an intercept **and** both dummies, you have perfect collinearity because:

$$1 = \text{nunion} + \text{husunion}$$

So one regressor is a perfect linear combination of the others (the intercept is a column of ones). R will drop a regressor and report NA for one coefficient.

```
lm3 <- lm(husearns ~ nunion + husunion, data = cps91)
summary(lm3)
```

```
##
## Call:
## lm(formula = husearns ~ nunion + husunion, data = cps91)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -595.92 -240.92  -68.92  149.08 1307.08
##
## Coefficients: (1 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  616.3330     11.3830   54.145  <2e-16 ***
## nunion       -0.4087     12.9924   -0.031    0.975
## husunion           NA           NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 353.4 on 4146 degrees of freedom
## (1486 observations deleted due to missingness)
## Multiple R-squared:  2.386e-07, Adjusted R-squared:  -0.000241
## F-statistic: 0.0009894 on 1 and 4146 DF,  p-value: 0.9749
```

Takeaway: when you have G categories, you include: - an intercept + $G - 1$ dummies, **or** - no intercept + all G dummies.

2. Robust standard errors (White / HC)

2.1 What the problem is asking (plain English)

The usual OLS standard errors assume homoskedasticity (constant variance of u given x).

If heteroskedasticity is present, OLS coefficients are still unbiased/consistent (under the usual assumptions), but the **usual** standard errors can be wrong. Robust (White/HC) standard errors fix the standard error calculation.

In this tutorial, we compute robust standard errors for `lm1`.

2.2 Step-by-step plan (how to solve it)

- 1) Fit the OLS model `lm1`.
- 2) Compute a robust variance-covariance matrix using `sandwich::vcovHC(lm1, type="HC0")`.

- 3) Extract robust standard errors via `sqrt(diag(vcov_robust))`.
- 4) Use `lmtest::coeftest()` to print coefficient tests using robust SE.
- 5) (HW) Compute the robust variance matrix “by hand” using the sandwich formula and verify it matches `vcovHC()`.

2.3 Common pitfalls

- **Forgetting that robust SE change t statistics and p-values but not coefficients.**
- **Using the wrong HC type:** HC0 is the original White; HC1 is a small-sample correction (often a good default in practice).
- **Mixing up variance and standard error:** SE is the square root of the variance (diagonal element).

2.4 Module: What are `vcovHC()` and `coeftest()`? (first-time tools)

`sandwich::vcovHC()`

`vcovHC()` computes a **heteroskedasticity-consistent** covariance matrix for the OLS coefficients.

The generic White “sandwich” estimator is:

$$\widehat{\text{Var}}(\hat{\beta}) = (X'X)^{-1} \left(\sum_{i=1}^n \hat{u}_i^2 x_i x_i' \right) (X'X)^{-1}.$$

- `type="HC0"` corresponds closely to the expression above.
- Other types (HC1, HC2, HC3, ...) make small-sample or leverage adjustments.

`lmtest::coeftest()`

`coeftest()` prints coefficient tests and allows you to **plug in a custom covariance matrix** (like a robust one).

That means you can keep the same OLS coefficients but replace the standard errors used for inference.

2.5 Robust SE using `vcovHC()` + `coeftest()`

```
robust_vcov <- vcovHC(lm1, type = "HC0")
```

```
# Robust standard errors for each coefficient
sqrt(diag(robust_vcov))
```

```
## (Intercept)    husunion
##      6.674769    10.760758
```

```
# Use robust VCOV inside coeftest.
# df = Inf uses the large-sample (normal) reference distribution.
coeftest(lm1, df = Inf, vcov = robust_vcov)
```

```
##
## z test of coefficients:
##
##              Estimate Std. Error z value Pr(>|z|)
```

```
## (Intercept) 615.92431    6.67477  92.276   <2e-16 ***
## husunion    0.40868    10.76076   0.038   0.9697
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

2.6 HW (complete): compute White (HC0) robust SE “by hand”

We will manually construct the sandwich estimator:

$$\widehat{\text{Var}}(\hat{\beta}) = (X'X)^{-1} X' \widehat{\Omega} X (X'X)^{-1}, \quad \widehat{\Omega} = \text{diag}(\hat{u}_1^2, \dots, \hat{u}_n^2).$$

```
# Design matrix X and residuals uhat
X <- model.matrix(lm1)      # includes intercept column automatically
uhat <- resid(lm1)

# Sandwich pieces
XtX_inv <- solve(t(X) %*% X)
meat <- t(X) %*% (diag(uhat^2)) %*% X

vcov_hc0_manual <- XtX_inv %*% meat %*% XtX_inv

# Compare manual vs vcovHC
max_abs_diff <- max(abs(vcov_hc0_manual - robust_vcov))
max_abs_diff
```

```
## [1] 1.058709e-12
```

Robust SE comparison (manual vs vcovHC):

```
cbind(
  se_manual = sqrt(diag(vcov_hc0_manual)),
  se_vcovHC = sqrt(diag(robust_vcov))
)
```

```
##           se_manual se_vcovHC
## (Intercept)  6.674769  6.674769
## husunion    10.760758 10.760758
```

Wrap-up

- Dummy variables: choose a base category (intercept + one dummy) OR remove intercept and include all category dummies.
- Dummy variable trap: intercept + all dummies creates perfect collinearity.
- Robust SE: `vcovHC()` changes standard errors (and inference), not the OLS coefficient estimates.
- `coeftest()` is a clean way to report robust t tests without rewriting the regression output.